

TudatPy JSON Interface

Code Structure, Contract Semantics, and Data Flow

Architecture report for the contract-driven settings interface

July 22, 2026

Abstract

The JSON interface is a settings-construction layer. JSON contracts define the permitted data model; the generic loader validates JSON values against those contracts and calls the same public TudatPy settings factories used by ordinary Python applications. It does not serialize runtime objects, propagate dynamics, or maintain a second hand-written implementation of each settings family. This report documents the implemented code structure, the flow of linked JSON data, and the end-to-end perturbed-satellite regression test.

1 Architectural rules

The implementation follows five rules:

1. **Contracts are authoritative.** Factory names, argument names, types, optional defaults, and temporarily unsupported inputs live in JSON contract files.
2. **The loader is generic.** Nested settings are recognized from dotted contract types and recursively constructed without module-specific conversion code.
3. **Factories are public TudatPy API.** A JSON factory name maps to a callable exposed by an identifiable `environment_setup` or `propagation_setup` module.
4. **JSON contains settings data only.** Live runtime objects such as `SystemOfBodies` are passed as context after environment creation.
5. **References compose files.** An object containing only `"$ref"` is replaced by the referenced JSON value, with paths resolved relative to the file containing the reference.

2 Code and file structure

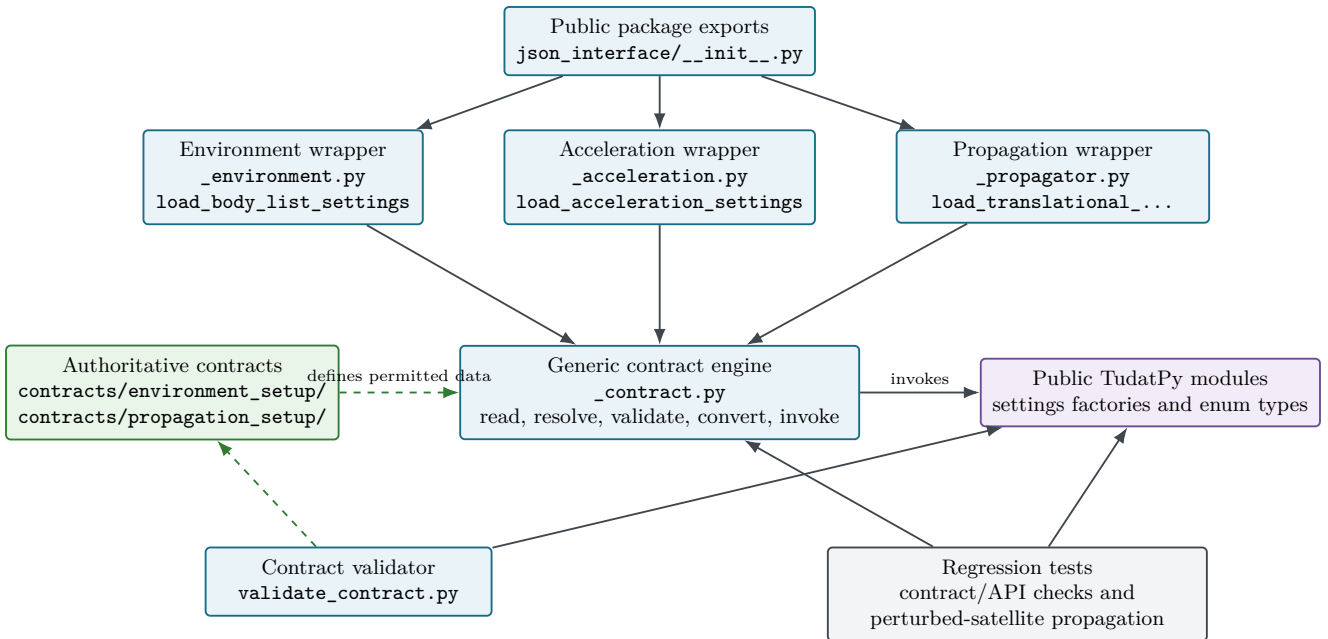


Figure 1: Implemented module structure. Solid arrows represent Python calls; dashed green arrows represent contract-governed validation.

2.1 Responsibilities

Component	Responsibility
<code>_contract.py</code>	Reads JSON, resolves references, parses type expressions, validates defaults and factory signatures, converts values, and invokes factories.
<code>_environment.py</code>	Extracts the global frame and injects it as context when creating the linked per-body settings, then returns <code>BodyListSettings</code> .
<code>_acceleration.py</code>	Selects the acceleration contract for the standard nested body/exerting-body acceleration-settings mapping.
<code>_propagator.py</code>	Injects the live body system so the public composition factory can create acceleration models from JSON acceleration settings.
<code>validate_contract.py</code>	Compares arbitrary contracts, or all packaged contracts, with the first exposed signatures of their public TudatPy factories.
<code>contracts/</code>	Shared data-model definitions. Editing a contract changes what the generic loader accepts; no parallel factory-specific loader table is required.

3 Generic settings-construction pipeline

For every loader call, the same pipeline is used:

1. `read_json_object` decodes the root file and recursively replaces relative `$ref` objects.
2. `load_contract` validates the contract structure, all type expressions, optional defaults, unsupported declarations, and factory existence.
3. `_convert_settings_tree` preserves structural dictionaries while recognizing one-key factory definitions at the leaves.
4. `create_settings_object` checks missing, unknown, and unsupported arguments; merges operational defaults; and converts every supported value.
5. The selected callable is retrieved from the public TudatPy module and invoked with converted JSON arguments plus any explicitly supplied runtime context.

3.1 Contract entry semantics

```
"spherical_harmonic_gravity": {  
  "properties": {  
    "maximum_degree": "int",  
    "maximum_order": "int"  
  }  
}
```

All properties are required unless their name appears in **optional**. A finite optional value is passed explicitly to the Tudat factory; a **null** default delegates to the factory default when JSON cannot represent that default reliably. An input in **unsupported** remains visible in the shared contract, but cannot be supplied. If it is required, that factory cannot yet be constructed from JSON; if it is optional and omitted, Tudat uses its own default.

4 End-to-end data flow

The perturbed-satellite test has two top-level settings inputs. Both can include arbitrarily nested relative references, but the Python test loads only these two root documents.

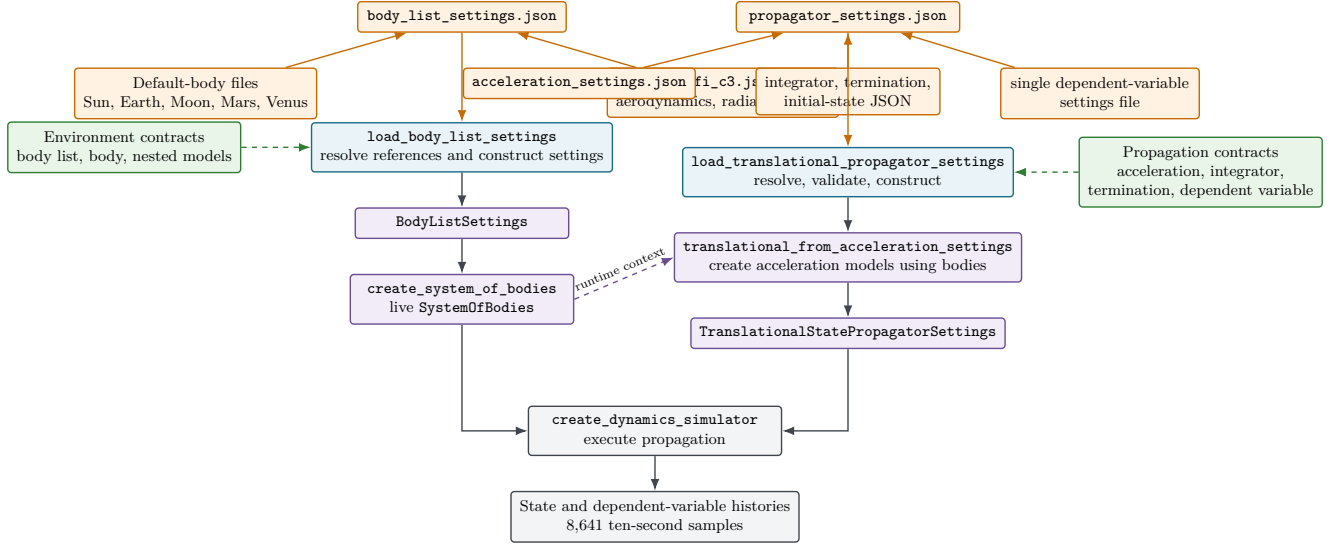


Figure 2: Data flow in the JSON-only perturbed-satellite test. Orange arrows are JSON/reference flow, dashed green arrows are contract validation, dashed purple is the live body context, and solid dark arrows are object/runtime flow.

5 The explanatory end-to-end test

The test is located at `tests/test_tudatpy/json_interface/perturbed_satellite_orbit/`. Its Python body deliberately contains no direct settings-factory calls:

```
def test_perturbed_satellite_orbit_from_json():
    # Runtime initialization, not settings construction.
    spice.load_standard_kernels()

    body_list_settings = load_body_list_settings(
        settings_directory / "body_list_settings.json"
    )
    bodies = environment_setup.create_system_of_bodies(body_list_settings)

    propagator_settings = load_translational_propagator_settings(
        settings_directory / "propagator_settings.json", bodies
    )

    dynamics_simulator = simulator.create_dynamics_simulator(
        bodies, propagator_settings
    )

    assert len(dynamics_simulator.propagation_results.state_history) == 8641
```

5.1 Environment input

`body_list_settings.json` declares the Earth-centered J2000 global frame and references six per-body files. The five celestial bodies request Tudat defaults. `body_settings/delfi_c3.json` contains all Delfi-C3 environment properties in one place:

- constant aerodynamic coefficients with reference area 0.035 m^2 and force coefficients $[1.2, 0, 0]$;
- a cannonball radiation-pressure target with coefficient 1.2 and Earth occulting the Sun;
- constant rigid-body properties with mass 2.2 kg.

The environment wrapper propagates the root global frame into the construction of every default body. It returns `BodyListSettings`; ordinary TudatPy then creates the live body system.

5.2 Propagation input

`propagator_settings.json` references separate files for:

File	Contents
<code>acceleration_settings.json</code>	Spherical-harmonic Earth gravity to degree and order 5, Earth aerodynamics, Sun radiation pressure, and point-mass gravity from Sun, Moon, Mars, and Venus.
<code>integrator_settings.json</code>	Fixed-step RK4 with a ten-second step.
<code>termination_settings.json</code>	Time termination at the end of the one-day interval.
<code>initial_state_from_tle.json</code>	Cartesian state derived from the TLE used by the ordinary perturbed-satellite example.
<code>dependent_variable_settings.json</code>	Total acceleration, Keplerian state, latitude, longitude, and the norms of each individual acceleration contribution.

The propagation wrapper supplies the live body system only as runtime context. The public `translational_from_acceleration_settings` factory creates acceleration models and then constructs ordinary translational propagator settings. The simulator call is therefore unchanged from a normal TudatPy application.

5.3 What the assertion proves

The expected 8,641 samples follow from an inclusive one-day interval sampled at ten-second integration steps:

$$\frac{86,400\text{ s}}{10\text{ s}} + 1 = 8,641.$$

Reaching this assertion demonstrates that all linked files were resolved, every nested settings definition passed its contract, public Tudat factories created the environment and propagation settings, acceleration models were composed with the runtime bodies, and the resulting settings supported a complete propagation.

6 Validation and failure boundaries

The regression suite separates structural correctness from end-to-end execution:

1. **Contract/API parity:** every contract factory exists and its required/optional argument names match the first public signature.
2. **Documentation parity:** every contracted factory is present in the generated API documentation sources.
3. **Type enforcement:** wrong scalar types, missing inputs, unknown inputs, unknown factories, and non-finite numbers are rejected before the factory call.
4. **Default semantics:** finite contract defaults are operational; null and optional unsupported inputs delegate to Tudat defaults.
5. **Reference semantics:** relative references work recursively and circular references are rejected.
6. **Runtime composition:** body settings create the system of bodies, and those bodies are injected only where acceleration-model construction requires them.
7. **Execution:** the perturbed-satellite test completes the one-day propagation and checks the full sample count.

7 Extension path

For an ordinary additional `environment_setup` or `propagation_setup` settings family, the intended extension is:

1. add or update its authoritative JSON contract;
2. ensure each factory is public, documented, and has a finite named-input interface;
3. use dotted property types to connect settings families;
4. mark inputs outside the supported settings layer explicitly as `unsupported`;
5. run the generic validator and relevant end-to-end tests.

No new conversion dispatch table is required. A thin wrapper is justified only when the outer document has a domain-specific shape or construction requires a live runtime object that cannot belong in JSON.